

Documentación de los componentes de Admin (Adrián Chicano García)

1.Componente: Clients

El componente de gestión de clientes se encarga de administrar la cartera de entidades donde se realizan las paradas logísticas, garantizando en todo momento la integridad y validez de los datos introducidos.

Estructura Visual (HTML):

A nivel de interfaz, se ha implementado el uso de property binding ([disabled]) para controlar dinámicamente el estado del botón de envío del formulario. Esta decisión permite bloquear la interacción del usuario cuando los datos no cumplen las condiciones de validación, evitando así peticiones innecesarias a la API.

Este enfoque responde al principio de validación en cliente, mejorando la eficiencia del sistema y reduciendo la carga en el backend.

```
<div class="modal-actions">
  <!-- El botón se deshabilita si los datos introducidos no cumplen las reglas -->
  <button type="submit" class="btn-guardar" [disabled]="clientForm.invalid">
    {{ isEditing ? 'Actualizar' : 'Guardar' }}
  </button>
</div>
```

Lógica de Negocio (TypeScript):

En la capa lógica, se emplean expresiones regulares (RegEx) como mecanismo de validación estricta para campos como el número de teléfono, exigiendo un formato exacto de nueve dígitos.

Esta validación previa a la persistencia garantiza la consistencia de los datos y evita la introducción de información errónea en la base de datos, reduciendo la necesidad de validaciones posteriores.

```
initForm() {
  const telefonoRegex = '^[0-9]{9}$'; // Obliga a que sean exactamente 9 valores numéricos
  this.clientForm = this.fb.group({
    nombre: ['', [Validators.required, Validators.minLength(3)]],
    telefono: ['', [Validators.pattern(telefonoRegex), Validators.required]]
  });
}
```

Estilos y UX (SCSS):

El diseño se basa en el uso de modales con superposición oscurecida (rgba) y centrado mediante Flexbox. Este patrón permite enfocar la atención del usuario en la tarea activa, siguiendo principios de diseño centrado en el usuario (UX).

```
.modal-overlay {  
  position: fixed;  
  top: 0; left: 0; width: 100%; height: 100%;  
  background: rgba(0, 0, 0, 0.6); /* Oscurece el fondo de la aplicación */  
  display: flex; justify-content: center; align-items: center;  
  z-index: 2000; /* Asegura que el formulario quede por encima de todo */  
}
```

Pruebas Realizadas:

Se diseñaron escenarios de prueba orientados a validar tanto el comportamiento funcional como los casos límite, incluyendo la introducción de formatos inválidos en teléfonos y direcciones, verificando el correcto bloqueo del sistema ante datos incorrectos.

Gestión de Clientes

Listado de todos los clientes.

[Nuevo Cliente](#)

Nombre	Dirección	Acciones	
Cliente Puerta del Sol	Calle Mayor 1, 28013 Madrid	Editar	Eliminar
Cliente Retiro	Paseo de García Lorca 2, 28009 Madrid	Editar	Eliminar
Cliente Salamanca	Calle de Serrano 45, 28001 Madrid	Editar	Eliminar
Cliente Moncloa	Calle de la Princesa 70, 28008 Madrid	Editar	Eliminar
Cliente Chamberí	Calle de Santa Engracia 93, 28010 Madrid	Editar	Eliminar

Nuevo Cliente

Nombre

Dirección

Teléfono

[Cancelar](#)[Guardar](#)

2.Componente: Routes

Este componente es uno de los más complejos del sistema, ya que se encarga de la creación y gestión de rutas logísticas optimizadas.

Estructura Visual (HTML):

En la interfaz, se facilita la selección de clientes mediante variables locales en el DOM, lo que permite capturar el valor seleccionado y enviarlo directamente a la función correspondiente sin necesidad de intermediarios.

```
<div class="add-stop">
  <select #clientSelect>
    <option value="" disabled selected>Añadir cliente...</option>
    <option *ngFor="let c of allClients" [value]="c.id">{{ c.nombre }}</option>
  </select>
  <button class="btn-add" (click)="addStop(clientSelect.value)">Añadir</button>
</div>
```

Lógica de Negocio (TypeScript):

Se ha implementado un algoritmo de reindexación que, tras la eliminación de una parada, recalcula automáticamente el orden de las restantes mediante un bucle forEach.

Esta solución garantiza la consistencia de estructuras secuenciales, evitando errores en la numeración y manteniendo la integridad lógica de las rutas.

```
removeStop(index: number) {
  this.paradas.splice(index, 1); // Extrae la parada seleccionada
  // Recorre las rutas restantes y recalcula el número de orden (1, 2, 3...)
  this.paradas.forEach((s, i) => s.orden = i + 1);
}
```

Estilos y UX (SCSS):

Para evitar problemas de usabilidad en rutas extensas, se ha limitado el tamaño de los modales mediante max-height y se ha habilitado scroll interno (overflow-y: auto), aplicando principios de diseño adaptable.

```
.lista-paradas {
  list-style: none;
  max-height: 200px;
  overflow-y: auto; /* Genera una barra de desplazamiento si excede el tamaño */
  border: 1px solid #eee;
}
```

Pruebas Realizadas:

Se validó el comportamiento dinámico del sistema ante operaciones de inserción y eliminación de paradas, comprobando la correcta actualización del orden y la estabilidad del sistema.

Nueva Ruta

Nombre de la Ruta

Paradas (Clientes)

▼
Añadir

1	Cliente Puerta del Sol	×
2	Cliente Retiro	×
3	Cliente Moncloa	×

Cerrar
Crear Ruta

Rutas Maestras

Visualiza las rutas creadas, añade nuevas o modifica las existentes.

Nueva Ruta

Ruta 1
5 paradas registradas
Ver / Editar

Madrid Sur
2 paradas registradas
Ver / Editar

Puebas
3 paradas registradas
Ver / Editar

3.Componente: Planning

El módulo de planificación permite asignar rutas a trabajadores en fechas concretas, siendo clave en la operativa del sistema.

Estructura Visual (HTML):

Se utiliza el atributo nativo min de HTML5 en los campos de fecha para restringir la selección a días actuales o futuros, implementando una primera capa de validación directamente en la interfaz.

```
<div class="form-group">
  <label for="date">Fecha de la Jornada</label>
  <!-- [min] evita que el navegador deje seleccionar días anteriores a "today" -->
  <input type="date" id="date" formControlName="fecha" [min]="today">
</div>
```

Lógica de Negocio (TypeScript):

Se incorpora una validación adicional en la lógica de negocio que verifica la fecha antes de enviarla al servidor. En caso de detectar una fecha inválida, el flujo se interrumpe.

Este enfoque sigue el principio de defensa en profundidad, asegurando la integridad de los datos en múltiples capas.

```
planify() {
  // Regla de negocio estricta: No se permite la planificación retroactiva
  if (this.journeyForm.value.fecha < this.today) {
    alert('¡Error! No puedes elegir una fecha pasada.');
```

Estilos y UX (SCSS):

Se han mejorado los indicadores de foco mediante box-shadow, sustituyendo los estilos nativos del navegador por elementos visuales más claros, mejorando la accesibilidad.

```
select:focus, input:focus {
  outline: none;
  border-color: #007bff;
  box-shadow: 0 0 0 2px rgba(0, 123, 255, 0.25); /* Aura azul estándar (Glow) */
}
```

Pruebas Realizadas: se simuló la asignación de jornadas en fechas pasadas, verificando que el sistema bloquea esta acción tanto desde la interfaz como desde la lógica interna.



Planificación de Jornadas

Asigna una ruta a un trabajador para un día determinado.

Seleccionar Trabajador

Pedro Picapiedra

Seleccionar Ruta

Puebas

Fecha de la Jornada

17/04/2026

Asignar Jornada

4.Componente: JourneyList

Este componente actúa como un panel de control para la visualización y seguimiento de las jornadas planificadas.

Estructura Visual (HTML):

Se emplea attribute binding para inyectar el estado de cada jornada en atributos personalizados (data-estado), permitiendo la aplicación de estilos dinámicos mediante CSS.

```
<div class="meta">
  <!-- Inyección de un atributo personalizado [attr.data-estado] -->
  <div class="Estado" [attr.data-estado]="j.estado">{{ j.estado }}</div>
  <div>
    <h3 class="trabajador">{{ j.trabajador }}</h3>
    <small class="email">{{ j.email_trabajador }}</small>
  </div>
</div>
```

Lógica de Negocio (TypeScript):

La lógica se basa en la gestión asíncrona de datos, incorporando un sistema de control de carga que evita el renderizado de información incompleta.

Este enfoque mejora la experiencia de usuario y garantiza la coherencia visual de la interfaz.

```
loadData() {
  this.loading = true; // Activa animaciones de carga en la vista
  // Recupera el estado actual de la flota
  this.api.getJourneys().subscribe({
    next: (res) => {
      this.journeys = res;
      this.loading = false;
    }
  });
}
```

Estilos y UX (SCSS):

Se utilizan elementos visuales tipo “badge” para representar los estados, facilitando la interpretación rápida de la información mediante recursos visuales claros.

```
.Estado {  
  padding: 0.2rem 0.6rem;  
  background: #e9ecef;  
  color: #000;  
  border-radius: 4px;  
  font-size: 0.75rem;  
  font-weight: 600;  
  text-transform: uppercase;  
}
```

Pruebas Realizadas:

Se comprobó la correcta actualización de la interfaz ante cambios de estado provenientes del servidor, validando la reactividad del sistema.

Jornadas Planificadas

Listado de todas las rutas asignadas a trabajadores.

FINALIZADA

Juan Operador
worker@mock.com
Ruta: Ruta 1 | Fecha: 2026-04-15

Eliminar Jornada

FINALIZADA

Juan Operador
worker@mock.com
Ruta: Ruta 1 | Fecha: 2026-04-15

Eliminar Jornada

CREADA

Pedro Picapiedra
pedro-picapiedra@mock.com
Ruta: Ruta 1 | Fecha: 2026-04-14

Eliminar Jornada

5.Componente: Workers

Este módulo permite la gestión de los trabajadores del sistema, incluyendo sus credenciales y datos asociados.

Estructura Visual (HTML):

Se implementa un renderizado dinámico mediante *ngFor, permitiendo la visualización eficiente de listas de usuarios y su interacción directa mediante eventos (click).

```
<!-- Listado dinámico con directiva estructural *ngFor -->
<tr *ngFor="let user of users">
  <td>{{ user.nombre }}</td>
  <td>{{ user.email }}</td>
  <td class="derecha">
    <button class="Editar" (click)="openModal(user)">Editar</button>
  </td>
</tr>
```

Lógica de Negocio (TypeScript):

Se ha desarrollado un sistema de validación dinámica que adapta las reglas del formulario según el contexto (creación o edición), evitando duplicidad de lógica.

```
openModal(user?: User) {
  if (user) {
    // Al editar un trabajador existente, cambiar la contraseña es opcional
    this.userForm.get('password')?.setValidators([Validators.minLength(6)]);
  } else {
    // Al crear un nuevo trabajador, la contraseña es obligatoria
    this.userForm.get('password')?.setValidators([Validators.required, Validators.minLength(6)]);
  }
  this.userForm.get('password')?.updateValueAndValidity(); // Aplica la regla
}
```

Estilos y UX (SCSS):

Se incorporan indicadores visuales en acciones críticas, como la eliminación de usuarios, utilizando cambios de color para advertir al usuario y prevenir errores.

```
button.Eliminar:hover {  
  background: #f8d7da;  
  border-color: #f5c6cb;  
  color: #721c24; /* El botón se pone rojo al pasar el ratón indicando peligro */  
}
```

Pruebas Realizadas:

Se validó el funcionamiento completo del ciclo CRUD, asegurando la actualización dinámica de la interfaz y el correcto comportamiento de las validaciones.

Nuevo Trabajador

Nombre Completo

Ej: Juan Pérez

Correo Electrónico

ejemplo@correo.com

Contraseña

Mínimo 6 caracteres

Cancelar

Registrar

Gestión de Trabajadores

Administra los accesos de tus empleados al sistema.

Nuevo Trabajador

Nombre	Email	Acciones	
Juan Operador	worker@mock.com	Editar	Eliminar
Maria Ruta	worker2@mock.com	Editar	Eliminar
Manolo el del bombo	manolo@gmail.com	Editar	Eliminar
Pedro Picapiedra	pedro-picapiedra@mock.com	Editar	Eliminar

6.Componente: Reports

Este módulo constituye la capa analítica del sistema, encargada de generar informes a partir de los datos recopilados.

Estructura Visual (HTML):

Se utilizan pipes de Angular para formatear datos en tiempo real, así como directivas condicionales (*ngIf) para mostrar información en función del estado de los resultados.

```
<tbody>
  <tr *ngFor="let day of reportData.days">
    <!-- Pinta la fecha en formato día/mes/año usando un Pipe -->
    <td>{{ day.fecha | date:'dd/MM/yyyy' }}</td>
    <!-- Si isPartial es verdadero, muestra la etiqueta de alerta roja -->
    <td>
      <span *ngIf="day.isPartial" class="badge bg-danger">Parcial</span>
      <span *ngIf="!day.isPartial" class="badge bg-primary">OK</span>
    </td>
  </tr>
</tbody>
```

Lógica de Negocio (TypeScript):

Se implementa un algoritmo de cálculo de rangos de fechas que gestiona correctamente los cambios de mes y año, garantizando la precisión de los informes generados.

Además, se realizan cálculos agregados que permiten obtener indicadores clave (KPIs), facilitando la toma de decisiones.

```
const y = this.filters.year;
const m = this.filters.month;

// Calcula correctamente el inicio del siguiente mes, incluso si es el cambio de año (diciembre a enero)
let mesSiguiente = m + 1 > 12 ? 1 : m + 1;
let añoSiguiente = m + 1 > 12 ? y + 1 : y;

// Asegura el formato 00 para peticiones SQL estables
const mSigFormateado = mesSiguiente < 10 ? '0' + mesSiguiente : mesSiguiente;
const fechaFin = `${añoSiguiente}-${mSigFormateado}-01`;
```

Estilos y UX (SCSS):

El diseño del panel se basa en CSS Grid, proporcionando una estructura flexible y adaptable que mejora la visualización de grandes volúmenes de información.

```
.cards-informes {
  display: grid;
  /* Las tarjetas se auto-escalan ocupando mínimo 200px de ancho */
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 1.5rem;
}
```

Pruebas Realizadas:

Se validaron los cálculos agregados y se diseñaron escenarios específicos para comprobar la detección de situaciones como “gestión parcial”, verificando la fiabilidad del sistema.

Informes de trabajo

Genera reportes detallados y cálculos de distancia por trabajador.

Trabajador
Pedro Picapiedra

Mes
Abril

Año
2026

Buscar

TOTAL KM
0 km

VISITAS / TIEMPO
3 / 0 min

ESTADO
Completo

Fecha	Distancia (Km)	Visitas (min)	Estado
17/04/2026	0.00 km	0 min	OK